

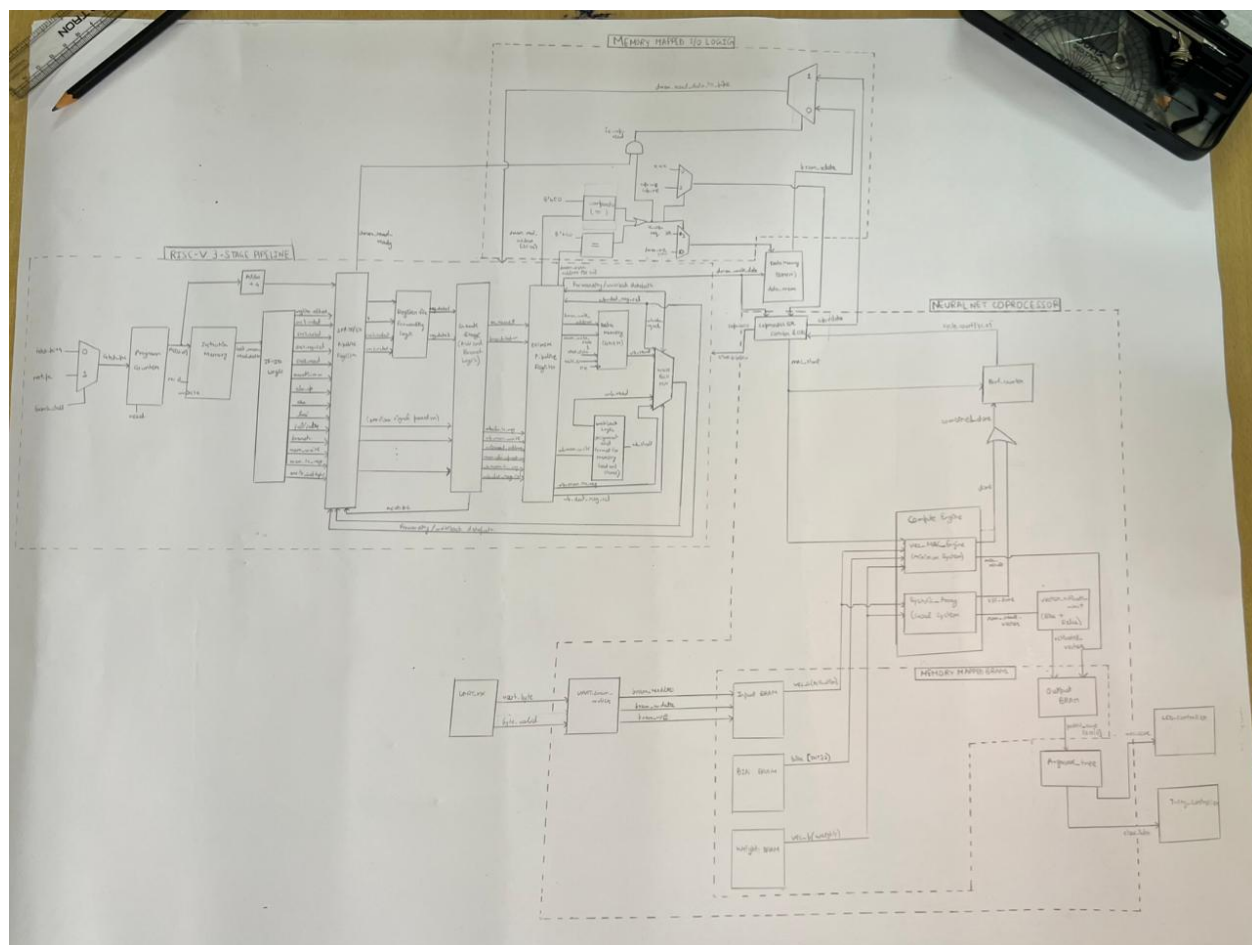
Block Diagram Report (3–4 pages)

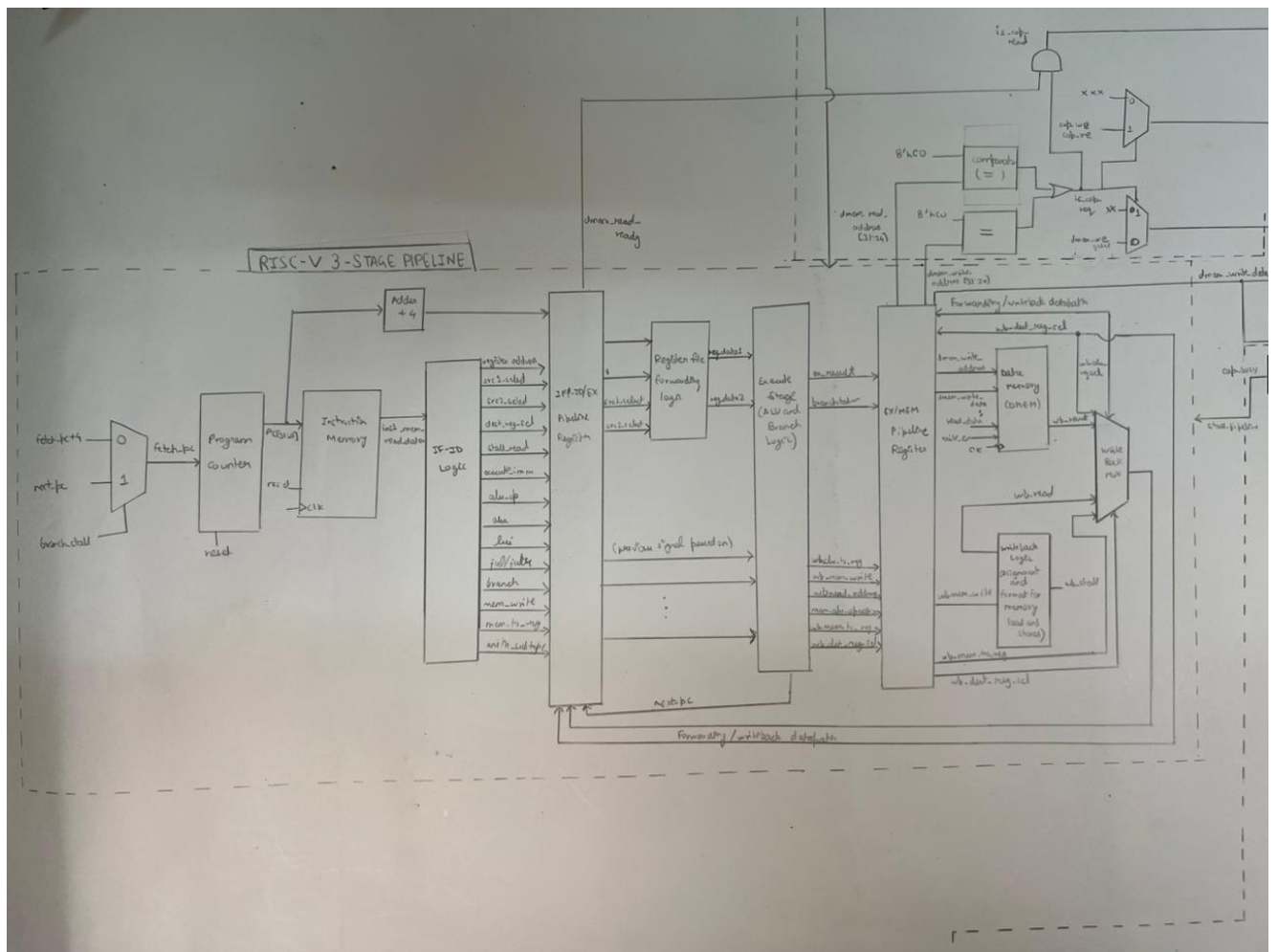
Title

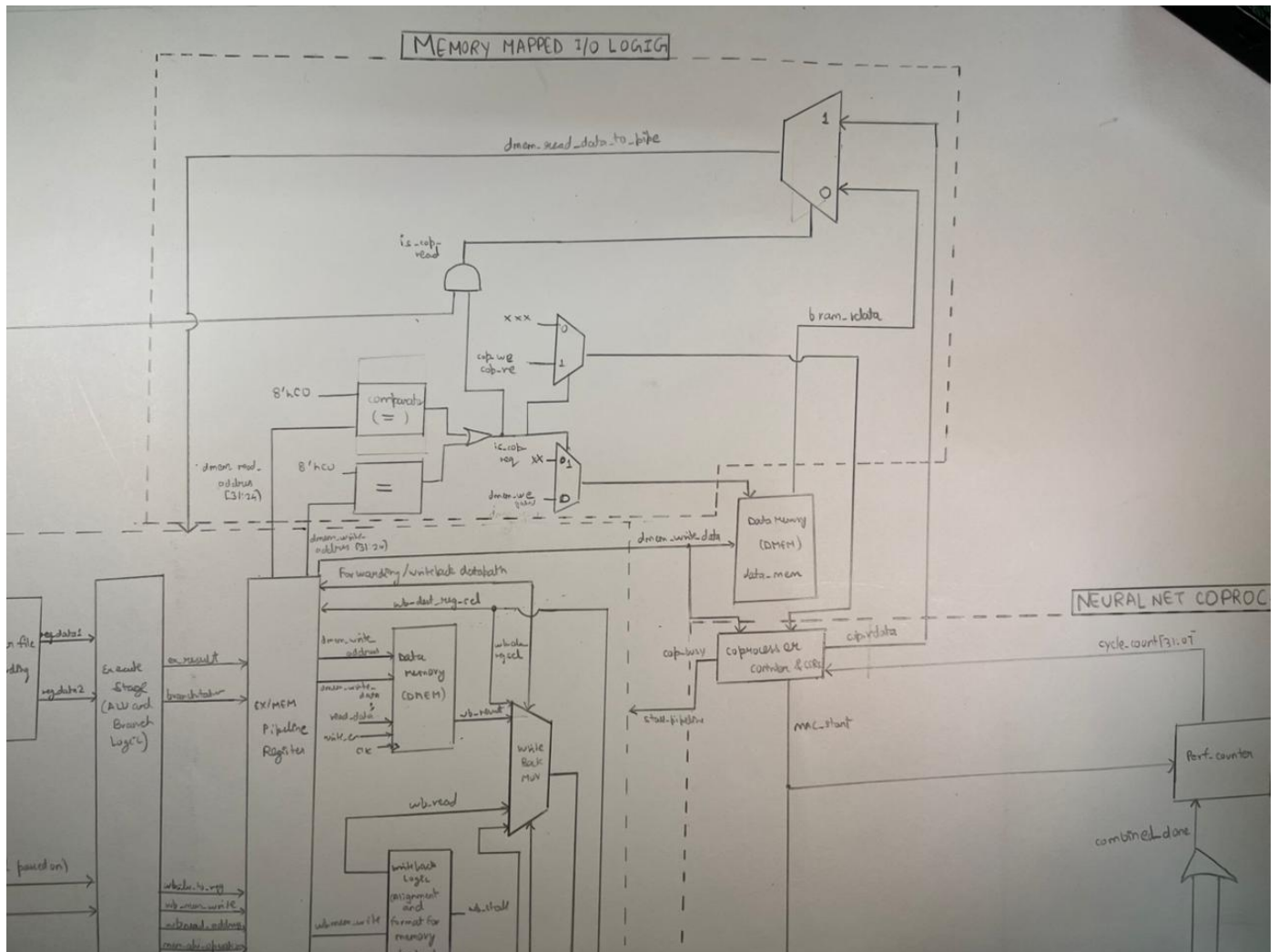
RISC-V NeuroCore: Edge Neural Network Inference via Systolic MAC Array

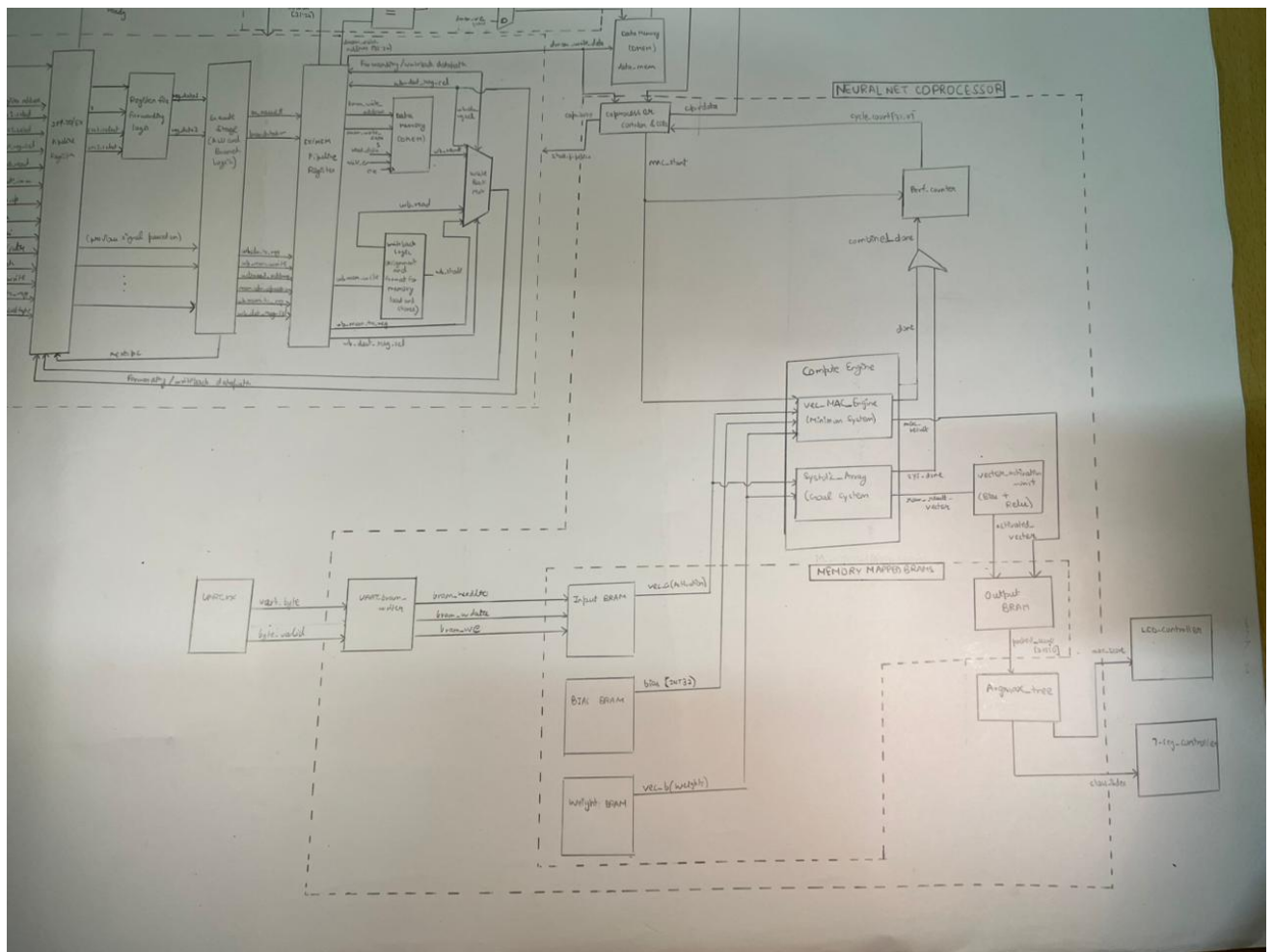
Group Number: 07

1. Top-Level Hardware Block Diagram (FULL PAGE)

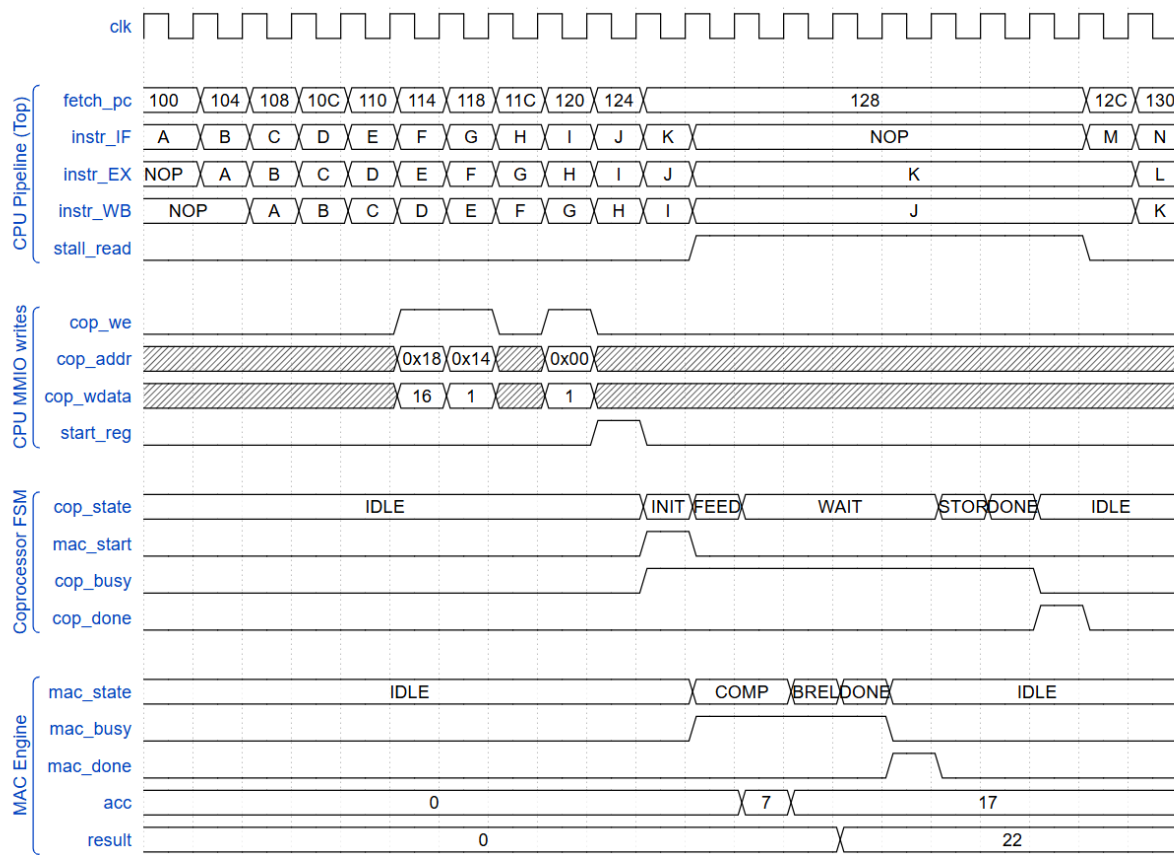








2. Timing Diagram (FULL PAGE)



CPU MMIO Writes & Execution Flow

Address	Label	Instruction	Register / MMIO Target
0x100	A	LUI x2, 0xC0000	x2 = 0xC000_0000 (coprocessor MMIO base)
0x104	B	ADDI x3, x0, 16	x3 = 16 (LAYER_COLS — 16 INT8 inputs per neuron)
0x108	C	ADDI x4, x0, 1	x4 = 1 (LAYER_ROWS — computing 1 neuron)
0x10C	D	SW x3, 0x18(x2)	CSR[0x18] = 16 (write LAYER_COLS to coprocessor)
0x110	E	SW x4, 0x14(x2)	CSR[0x14] = 1 (write LAYER_ROWS to coprocessor)
0x114	F	ADDI x5, x0, 1	x5 = 1 (START trigger value)
0x118	G	SW x5, 0x00(x2)	CSR[0x00] = 1 (FIRES COPROCESSOR)
0x11C	H	ADDI x0, x0, 0	NOP (slips through 1-cycle window)
PIPELINE FROZEN FROM HERE			
0x120	I	LW x6, 0x04(x2)	read STATUS register (first after resume)

0x124	J	LW x7, 0x20(x2)	read CYCLE_COUNT register (benchmark)
0x128	K	ADD x8, x6, x0	x8 = predicted class index

Hardware MAC Computation (Dot Product)

vec_a = [2, 3,-1, 4, 0, 1, 2,-2 | 1, 0, 3,-1, 2, 1, 0, 2]

vec_b = [1, 2, 3, 1, 5, 2, 1, 3 | 2, 1, 1, 4, 2, 3, 1, 1]

Batch 1 sum = $(2 \times 1) + (3 \times 2) + (-1 \times 3) + (4 \times 1) + (0 \times 5) + (1 \times 2) + (2 \times 1) + (-2 \times 3) = \mathbf{7}$

Batch 2 sum = $(1 \times 2) + (0 \times 1) + (3 \times 1) + (-1 \times 4) + (2 \times 2) + (1 \times 3) + (0 \times 1) + (2 \times 1) = \mathbf{10}$

acc = $7 + 10 = \mathbf{17}$

bias = $17 + \text{bias}(5) = \mathbf{22}$

ReLU = $\max(0, 22) = \mathbf{22}$

3. Module Summary (Table Only)

Module	Purpose	File	SW / HW ?
IF_ID	Instruction fetch + decode stage; immediate generation; id_ex pipeline register	IF_ID.v	Hardware (Verilog)
execute	ALU operations; branch/jump resolution; ex_mem_wb pipeline register	execute.v	Hardware (Verilog)
data_mem / instr_mem	Synchronous BRAM for instruction and data memory (4 KB each)	memory.v	Hardware (Verilog)
wb	Write-back stage; store data preparation; load byte/halfword formatting	wb.v	Hardware (Verilog)
pipe	Top-level pipeline integrator; register file; PC logic; stall register	pipeline.v	Hardware (Verilog)
top_fpga	FPGA top level; MMIO address decoder (0xC0xxxxx); stall injection into pipeline; module wiring	top_fpga.v	Hardware (Verilog)
coprocessor_top	Master CSR hub; 6-state FSM sequencing MAC engine across neurons; BRAM address control; cop_busy/done signals	coprocessor_top.v	Hardware (Verilog)
vec_mac_engine	8-lane parallel INT8 dot-product engine with fused ReLU+Bias; 4-state FSM; Vivado infers DSP48 slices	vec_mac_engine.v	Hardware (Verilog)
pe	Single Processing Element for systolic array; registered MAC: $acc_out = acc_in + a_in \times b_in$	pe.v	Hardware (Verilog)
systolic_array	2-D grid of PE instances for parallel multi-neuron dot products; diagonal data skewing	systolic_array.v	Hardware (Verilog)
argmax_tree	Combinational tournament-reduction tree; finds winning	argmax_tree.v	Hardware (Verilog)

Module	Purpose	File	SW / HW ?
	class index among 10 INT32 logits		
weight_bram	Dual-port BRAM (32-bit wide, 16 K depth) for INT8 weight storage; initialised from .mem file	weight_bram.v	Hardware (Verilog)
input_bram	Single-port BRAM for layer input activations (784 INT8 values for MNIST)	input_bram.v	Hardware (Verilog)
output_bram	Single-port BRAM for INT32 accumulator results; doubles as input for next layer	output_bram.v	Hardware (Verilog)
bias_bram	Single-port BRAM for INT32 per-neuron biases; initialised from .mem file	bias_bram.v	Hardware (Verilog)
uart_rx	8N1 UART receiver (115 200 baud, 100 MHz); 5-state FSM; 1-cycle byte_valid pulse	uart_rx.v	Hardware (Verilog)
uart_bram_writer	Packs 4 serial bytes → 32-bit word; writes sequentially into input_bram; asserts transfer_complete at 784 bytes	uart_bram_writer.v	Hardware (Verilog)
led_controller	Maps max logit score to 16-LED confidence bar (combinational)	led_controller.v	Hardware (Verilog)
seven_seg_controller	Multiplexed 4-digit 7-segment driver (1 kHz refresh); displays predicted digit class 0–9	seven_seg_controller.v	Hardware (Verilog)
perf_counter	32-bit cycle counter; starts on mac_start, stops on mac_done; value readable via CYCLE_COUNT CSR	perf_counter.v	Hardware (Verilog)
quantize_weights.py	Trains 784→128→10 MLP on MNIST; applies INT8 post-training quantisation; exports .mem and weights.h	python/quantize/	Software (Python)
generate_test_vectors.py	Generates 500+ random INT8 vector pairs + NumPy	python/quantize/	Software (Python)

Module	Purpose	File	SW / HW ?
	expected outputs for testbench validation		
numpy_golden_model.py	Reference model (int8_mac, systolic_layer, argmax) used to validate all hardware outputs	sim/golden/	Software (Python)
host_gui.py	Tkinter canvas; captures hand-drawn digit; sends 784 bytes over pyserial UART; displays result	python/host/	Software (Python)
dot_product_bench.c	C workload 1: SW vs HW dot-product timing; reads CYCLE_COUNT CSR and prints speedup	sw/c/	Software (C / RISC-V)
layer1_test.c	C workload 2: configures and fires coprocessor for 784×128 layer 1; reads back 128 outputs	sw/c/	Software (C / RISC-V)
layer2_test.c	C workload 3: reconfigures for 128×10 layer 2; reads back 10 logit scores	sw/c/	Software (C / RISC-V)
full_inference.c	C workload 4: end-to-end L1→L2→argmax; prints predicted digit and per-layer cycle counts	sw/c/	Software (C / RISC-V)
cycle_counter_bench.c	C workload 5: prints SW vs HW cycle-count table for full layer; demonstrates ≥50× speedup	sw/c/	Software (C / RISC-V)
mmio_test.s	Assembly smoke-test: raw SW/LW to CSR addresses; verifies stall and STATUS before C toolchain ready	sw/asm/	Software (Assembly)

4. Hardware Resource Estimates

Resource	Max Available (XC7A100T)	Estimated Usage	% Utilisation
BRAM (36Kb tiles — RAMB36)	135	20 RAMB36 (weight_bram takes ~15 blocks; IMEM, DMEM, input, output, and bias take ~1 block each).	~15%
DSP48E1 slices	240	~72 DSPs (8 for vec_mac_engine 8-lane INT8 MAC; 64 for systolic_array 8×8 PE grid, 1 DSP per PE multiply)	~30%
LUTs	63,400	~14,000 (FSMs + MMIO decoder + argmax tree + UART + 7-seg + pipeline logic)	~22%
Flip-Flops (FF)	126,800	~10,000 (pipeline registers + BRAM address counters + UART shift register + CSR regs)	~8%

5. Verification Plans (≤ 6–8 lines total OR 3–4 bullets)

- vec_mac_engine testbench (tb_vec_mac_engine.v): 500+ randomised INT8 vector pairs generated by generate_test_vectors.py; hardware dot-product results compared line-by-line against numpy_golden_model.py. Edge cases include all-zero inputs, max-value saturation ($+127 \times +127$), and alternating-sign vectors to stress ReLU clamping.
- Module-level simulation: dedicated testbenches for argmax_tree (100 random logit sets vs np.argmax), uart_rx (byte-accurate 8N1 task driving rxd at 115 200 baud), and systolic_array (known 4×4 weight matrix, result verified after N+M-1 cycles).
- System simulation (tb_coprocessor_top.v): drive CSR writes via simulated SW instructions, assert START, wait for cop_done, read output BRAM and compare every output neuron against NumPy golden model.
- Software validation on FPGA: five C workloads compiled with -march=rv32im and executed on the physical FPGA; terminal output (UART) must match expected predicted class and cycle counts within $\pm 2\%$.

6. Gantt Chart

[illegible]

7. Risks & Challenges (≤ 5 bullets, 1 line each)

- Pipeline stall timing: `stall_read` is a registered version of `cop_busy` (1-cycle slip), meaning one extra instruction enters the pipeline before it freezes; must be handled carefully to avoid corrupting register state when the CPU dispatches to the coprocessor.
- Signed vs unsigned INT8 arithmetic: all MAC lane signals must be declared signed [7:0] in Verilog; omitting the signed keyword causes Vivado to infer unsigned multiplications, producing silently wrong dot-product results for negative weights.
- BRAM read latency: all BRAMs have 1-cycle synchronous read latency; the coprocessor FSM must present the read address one state before consuming the data, otherwise the first data word per neuron will be garbage.
- FPGA timing closure with wide bus: the 8-lane 64-bit `vec_a` / `vec_b` buses between the BRAM and MAC engine may fail timing at 100 MHz if routing is unfavourable; mitigation is to add a pipeline register stage between BRAM output and the DSP inputs.
- Simulation vs FPGA mismatch: uninitialized BRAM contents and reset sequencing differences are common causes of passing simulation but failing FPGA execution; ILA probes on `cop_state`, `mac_state`, and `acc` will be set up for FPGA-side debugging.

8. Response to TA feedback (for the project abstract)

Point 1 — How do you handle pipeline stalls and CPU–coprocessor synchronization?

In our project, the RISC-V CPU runs a C program (`full_inference.c`) that dispatches neural network layer computations to the coprocessor entirely through ordinary store-word (SW) instructions. The C code does: `COP[5] = 128` (`LAYER_ROWS`), `COP[6] = 784` (`LAYER_COLS`), then `COP[0] = 1` (`START`). The compiler compiles each of these into a SW instruction targeting addresses in the `0xC000_0000` range — there is no special ISA extension. The CPU has no idea it is talking to a coprocessor; it just sees a memory write.

Because the hardware auto-stalls the CPU via `cop_busy`, the C program does not need to waste cycles polling a status register. The CPU simply goes to sleep when `START` is written and wakes up exactly when the result is ready to be read. The stall freezes the CPU for exactly as long as the coprocessor needs, making the hardware acceleration completely transparent to the C program.

Mechanically, when the `START` write reaches the WB stage of the pipeline, the MMIO address decoder in `top_fpga.v` detects the `0xC0` prefix on `dmem_write_address` and asserts `is_cop_write`. The `coprocessor_top` module latches `start_reg = 1` on the next rising edge. The master FSM immediately leaves `S_IDLE` and `cop_busy` — which is defined combinatorially as `(state != S_IDLE)` — goes high. `cop_busy` is wired directly to the stall input of the pipeline module. Inside `pipeline.v`, stall is registered into `stall_read` on the next clock edge, which holds `fetch_pc` constant (PC stops incrementing) and freezes both pipeline registers `id_ex_reg` and `ex_mem_wb_reg` so no new instructions commit. The pipeline stays frozen for the entire duration of the MAC computation — for a 784-element dot product with 8 lanes this is

approximately $784/8 = 98$ accumulate cycles plus FSM overhead — before the stall releases and the CPU resumes exactly where it paused.

Point 2 — LUT, FF, BRAM, and DSP usage estimates; will the design fit the target FPGA?

Detailed estimates are provided in Section 4 above, targeting the Nexys A7 (Artix-7 XC7A100T). In summary: the design is estimated to consume approximately 20 of 135 available 36Kb BRAM tiles (RAMB36) (~15%), 72 of 240 DSP48E1 slices (30%), 14,000 of 63,400 LUTs (~22%), and 10,000 of 126,800 flip-flops (~8%). All four figures are comfortably within device limits, leaving headroom for the systolic array goal system and stretch goals. The design will fit on the target FPGA. These estimates will be confirmed and refined after the first Vivado implementation run, expected by the Apr 12 milestone.

Point 3 — More detailed timeline; per-module milestones (design / testbench / integration); owners; feasibility.

The Gantt chart in Section 6 lists 35 concrete, testable activities each with a named owner and date range. For every hardware module the schedule explicitly identifies three milestones: (i) implementation complete (RTL written), (ii) testbench passing (simulation verified against NumPy golden model), and (iii) FPGA integration ready (instantiated in `top_fpga.v` and synthesised without errors). The five software deliverables (C workloads, Python quantisation, host GUI) each have independent owners and dates that do not block the RTL path. All five team members carry parallel workstreams from week one: Varun owns the MAC and systolic datapaths; Avinash owns BRAM infrastructure and top-level integration; Samvit owns all software and quantisation; Shubham owns the UART path and host-side GUI; Ved owns the hardware display (7-segment), post-processing, verification infrastructure, and ILA setup. The schedule is structured so that the minimum system (single-layer FPGA demo) is achieved by Apr 12, the goal system (full MNIST end-to-end via UART) by Apr 19, and the final two weeks are reserved for stretch goals, bug-fixing, and the final report and video.
